MINI PROJECT

EX. NO: ____ STUDENT ACADEMIC PROGRESS AND EARLY SUPPORT

DATE: 08.10.2026

NAME: Swetha M

REGISTER NO.: 20524UCSC049

Aim

To develop a Student Academic Progress and Early Support system using SQL and PL/SQL to store, manage, and analyze student GPA, attendance, internal marks, and backlog information, and to identify students who need early academic support.

Software Used

- Oracle Database
- SQL
- PL/SQL

Problem Statement

Develop a student progress monitoring system to identify academic difficulties using GPA, attendance, internal marks, and backlog information.

Description

The system stores the student ID, name, school, sex, age, semester, study time, absences, three period grades (G1, G2, G3), internal marks, attendance percentage, backlogs, and GPA of 25 students in one Oracle table named STUDENT_PROGRESS.

SQL queries are used to retrieve and analyze student information, while PL/SQL functions and procedures are used to find the academic risk category of a student, retrieve the progress report of a selected student, generate a semester-wise support report, and handle exceptions.

Support rule (documented educational threshold): a student is flagged Needs Support when GPA is below 5.0, or attendance is below 75%, or the student has one or more backlogs. Otherwise the student is On Track. This is a teaching rule, not an official college policy.

Dataset Source

- Dataset: UCI Student Performance / Student Progress sample dataset
- Authors: Cortez and Silva (2008), UCI Machine Learning Repository
- Records used: 25 (12 in semester 1 and 13 in semester 2)

- UCI attributes used: school, sex, age, studytime, absences, G1, G2, G3
- Added fields: student_id (101 to 125, primary key), student_name (fictional), semester, internal_marks, attendance_pct, backlogs and gpa (= g3 / 2)
- The rows are a hand-prepared sample in the structure and value ranges of the UCI file, not rows copied from student-mat.csv.

Database Table

STUDENT_PROGRESS

Column Name	Data Type	Constraint
STUDENT_ID	NUMBER(5)	PRIMARY KEY
STUDENT_NAME	VARCHAR2(40)	NOT NULL
SCHOOL	VARCHAR2(2)	NOT NULL, IN ('GP','MS')
SEX	CHAR(1)	NOT NULL, IN ('F','M')
AGE	NUMBER(2)	NOT NULL, BETWEEN 15 AND 22
SEMESTER	NUMBER(1)	NOT NULL, BETWEEN 1 AND 8
STUDY_TIME	NUMBER(1)	NOT NULL, BETWEEN 1 AND 4
ABSENCES	NUMBER(3)	NOT NULL, DEFAULT 0, >= 0
G1, G2, G3	NUMBER(2)	NOT NULL, BETWEEN 0 AND 20
INTERNAL_MARKS	NUMBER(2)	NOT NULL, BETWEEN 0 AND 25
ATTENDANCE_PCT	NUMBER(5,2)	NOT NULL, BETWEEN 0 AND 100
BACKLOGS	NUMBER(2)	NOT NULL, DEFAULT 0, >= 0
GPA	NUMBER(3,1)	NOT NULL, BETWEEN 0 AND 10

TABLE CREATION

Oracle

```
CREATE TABLE student_progress (
  student_id NUMBER(5),
  student_name VARCHAR2(40) NOT NULL,
  school VARCHAR2(2) NOT NULL,
  sex CHAR(1) NOT NULL,
  age NUMBER(2) NOT NULL,
  semester NUMBER(1) NOT NULL,
  study_time NUMBER(1) NOT NULL,
  absences NUMBER(3) DEFAULT 0 NOT NULL,
  g1 NUMBER(2) NOT NULL,
  g2 NUMBER(2) NOT NULL,
  g3 NUMBER(2) NOT NULL,
```

```
  internal_marks NUMBER(2) NOT NULL,
  attendance_pct NUMBER(5,2) NOT NULL,
  backlogs NUMBER(2) DEFAULT 0 NOT NULL,
  gpa NUMBER(3,1) NOT NULL,
  CONSTRAINT pk_student_progress PRIMARY KEY (student_id),
  CONSTRAINT ck_sp_school CHECK (school IN ('GP','MS')),
  CONSTRAINT ck_sp_sex CHECK (sex IN ('F','M')),
  CONSTRAINT ck_sp_age CHECK (age BETWEEN 15 AND 22),
  CONSTRAINT ck_sp_semester CHECK (semester BETWEEN 1 AND 8),
  CONSTRAINT ck_sp_studytime CHECK (study_time BETWEEN 1 AND 4),
  CONSTRAINT ck_sp_absences CHECK (absences >= 0),
  CONSTRAINT ck_sp_grades CHECK (g1 BETWEEN 0 AND 20
    AND g2 BETWEEN 0 AND 20
    AND g3 BETWEEN 0 AND 20),
  CONSTRAINT ck_sp_internal CHECK (internal_marks BETWEEN 0 AND 25),
  CONSTRAINT ck_sp_attendance CHECK (attendance_pct BETWEEN 0 AND 100),
  CONSTRAINT ck_sp_backlogs CHECK (backlogs >= 0),
  CONSTRAINT ck_sp_gpa CHECK (gpa BETWEEN 0 AND 10)
);
```

Output




```
Table created.

0.05 seconds
```

Table Created

The STUDENT_PROGRESS table was created successfully with primary key, CHECK, DEFAULT, and NOT NULL constraints.

DATASET INSERTION

Column order: id, name, school, sex, age, semester, study_time, absences, g1, g2, g3, internal_marks, attendance_pct, backlogs, gpa.

```
INSERT INTO student_progress VALUES (101,'Anitha
R','GP','F',17,1,3,4,12,13,14,19,92.50,0,7.0);
INSERT INTO student_progress VALUES (102,'Bhavani
S','GP','F',16,1,2,10,9,10,10,14,78.00,1,5.0);
INSERT INTO student_progress VALUES (103,'Charulatha
K','MS','F',18,1,4,2,15,16,17,22,96.00,0,8.5);
```

```
INSERT INTO student_progress VALUES (104,'Deepika
M','GP','F',17,1,1,14,7,8,6,10,62.50,2,3.0);
INSERT INTO student_progress VALUES (105,'Elakkiya
P','GP','F',16,1,3,6,11,12,12,17,85.00,0,6.0);
INSERT INTO student_progress VALUES (106,'Farhana
B','MS','F',17,1,2,8,10,9,9,12,74.00,1,4.5);
INSERT INTO student_progress VALUES (107,'Gayathri
V','GP','F',18,1,3,0,14,15,15,20,100.00,0,7.5);
INSERT INTO student_progress VALUES (108,'Harish
T','GP','M',15,1,4,3,16,17,18,23,94.00,0,9.0);
INSERT INTO student_progress VALUES (109,'Imran
L','MS','M',16,1,2,12,8,8,7,11,68.00,1,3.5);
INSERT INTO student_progress VALUES (110,'Janani
C','GP','F',17,1,3,5,12,11,12,16,88.00,0,6.0);
INSERT INTO student_progress VALUES (111,'Kavitha
N','GP','F',16,1,2,9,10,11,11,15,79.50,0,5.5);
INSERT INTO student_progress VALUES (112,'Lakshmi
D','MS','F',18,1,3,1,13,14,14,19,97.50,0,7.0);
INSERT INTO student_progress VALUES (113,'Meena
A','GP','F',17,2,1,16,6,7,5,8,58.00,3,2.5);
INSERT INTO student_progress VALUES (114,'Naveen
J','GP','M',15,2,3,4,13,13,14,18,90.00,0,7.0);
INSERT INTO student_progress VALUES (115,'Oviya
K','MS','F',16,2,2,7,11,10,10,14,81.00,0,5.0);
INSERT INTO student_progress VALUES (116,'Priya
G','GP','F',17,2,4,2,17,18,19,24,98.00,0,9.5);
INSERT INTO student_progress VALUES (117,'Ramesh
H','GP','M',18,2,2,11,9,9,8,12,72.50,1,4.0);
INSERT INTO student_progress VALUES (118,'Saranya
M','MS','F',16,2,3,6,12,12,13,17,86.50,0,6.5);
INSERT INTO student_progress VALUES (119,'Thenmozhi
S','GP','F',17,2,3,3,14,14,15,20,93.00,0,7.5);
INSERT INTO student_progress VALUES (120,'Uma
P','GP','F',15,2,2,13,8,9,9,13,70.00,0,4.5);
INSERT INTO student_progress VALUES (121,'Vasanth
R','MS','M',17,2,3,5,12,13,13,18,87.00,0,6.5);
INSERT INTO student_progress VALUES (122,'Yamini
B','GP','F',16,2,1,15,7,6,6,9,64.00,2,3.0);
INSERT INTO student_progress VALUES (123,'Zeenath
F','MS','F',18,2,4,0,15,16,16,21,99.00,0,8.0);
INSERT INTO student_progress VALUES (124,'Abirami
C','GP','F',16,2,2,8,10,11,12,16,80.00,1,6.0);
```

```
INSERT INTO student_progress VALUES (125,'Bharath
K','GP','M',17,2,3,4,13,12,12,17,89.00,0,6.0);
COMMIT;
```

Output

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.01 seconds
```

Results Explain Describe Saved SQL History

```
Statement processed.

0.00 seconds
```

Records Inserted

A total of 25 student records (12 in semester 1 and 13 in semester 2) were inserted into the STUDENT_PROGRESS table.

```
SELECT COUNT(*) FROM student_progress;
```

Output

Results Explain Describe Saved SQL History

COUNT(*)
25

1 rows returned in 0.03 seconds CSV Export

SQL Query 1: Display Student Progress Records by Semester

```
SELECT student_id, student_name, semester, gpa, attendance_pct,
  internal_marks, backlogs
FROM student_progress
ORDER BY semester, student_id;
```

Output

Results Explain Describe Saved SQL History

STUDENT_ID	STUDENT_NAME	SEMESTER	GPA	ATTENDANCE_PCT	INTERNAL_MARKS	BACKLOGS
101	Anitha R	1	7	92.5	19	0
102	Bhavani S	1	5	78	14	1
103	Charulatha K	1	8.5	96	22	0
104	Deepika M	1	3	62.5	10	2
105	Elakkiya P	1	6	85	17	0
106	Farhana B	1	4.5	74	12	1
107	Gayathri V	1	7.5	100	20	0
108	Harish T	1	9	94	23	0
109	Imran L	1	3.5	68	11	1
110	Janani C	1	6	88	16	0
111	Kavitha N	1	5.5	79.5	15	0
112	Lakshmi D	1	7	97.5	19	0
113	Meena A	2	2.5	58	8	3
114	Naveen J	2	7	90	18	0
115	Oviya K	2	5	81	14	0
116	Priya G	2	9.5	98	24	0
117	Ramesh H	2	4	72.5	12	1
118	Saranya M	2	6.5	86.5	17	0
119	Thenmozhi S	2	7.5	93	20	0
120	Uma P	2	4.5	70	13	0
121	Vasanth R	2	6.5	87	18	0
122	Yamini B	2	3	64	9	2
123	Zeenath F	2	8	99	21	0
124	Abirami C	2	6	80	16	1
125	Bharath K	2	6	89	17	0

25 rows returned in 0.03 seconds CSV Export

SQL Query 2: Average GPA for Each Semester

```
SELECT semester, COUNT(*) AS total_students,
  ROUND(AVG(gpa), 2) AS avg_gpa
FROM student_progress
GROUP BY semester
ORDER BY semester;
```

Output

Results Explain Describe Saved SQL History

SEMESTER	TOTAL_STUDENTS	AVG_GPA
1	12	6.04
2	13	5.85

2 rows returned in 0.03 seconds CSV Export

SQL Query 3: Students with One or More Backlogs

```
SELECT student_id, student_name, semester, backlogs, gpa
FROM student_progress
WHERE backlogs >= 1
ORDER BY backlogs DESC, student_id;
```

Output

Results Explain Describe Saved SQL History

STUDENT_ID	STUDENT_NAME	SEMESTER	BACKLOGS	GPA
113	Meena A	2	3	2.5
104	Deepika M	1	2	3
122	Yamini B	2	2	3
102	Bhavani S	1	1	5
106	Farhana B	1	1	4.5
109	Imran L	1	1	3.5
117	Ramesh H	2	1	4
124	Abirami C	2	1	6

8 rows returned in 0.03 seconds CSV Export

SQL Query 4: Attendance Percentage and GPA in Descending Order of Attendance

```
SELECT student_id, student_name, attendance_pct, gpa
FROM student_progress
ORDER BY attendance_pct DESC, gpa DESC;
```

Output

Results | Explain | Describe | Saved SQL | History

STUDENT_ID	STUDENT_NAME	ATTENDANCE_PCT	GPA
107	Gayathri V	100	7.5
123	Zeenath F	99	8
116	Priya G	98	9.5
112	Lakshmi D	97.5	7
103	Charulatha K	96	8.5
108	Harish T	94	9
119	Thenmozhi S	93	7.5
101	Anitha R	92.5	7
114	Naveen J	90	7
125	Bharath K	89	6
110	Janani C	88	6
121	Vasanth R	87	6.5
118	Saranya M	86.5	6.5
105	Elakkiya P	85	6
115	Oviya K	81	5
124	Abirami C	80	6
111	Kavitha N	79.5	5.5
102	Bhavani S	78	5
106	Farhana B	74	4.5
117	Ramesh H	72.5	4
120	Uma P	70	4.5
109	Imran L	68	3.5
122	Yamini B	64	3
104	Deepika M	62.5	3
113	Meena A	58	2.5

25 rows returned in 0.03 seconds CSV Export

SQL Query 5: Count of Students Categorised as Needs Support or On Track

Threshold: Needs Support when GPA < 5.0 or attendance < 75% or backlogs >= 1.

```
SELECT support_status, COUNT(*) AS total_students
FROM (
  SELECT CASE
     WHEN gpa < 5.0 OR attendance_pct < 75 OR backlogs >= 1
     THEN 'Needs Support'
     ELSE 'On Track'
   END AS support_status
  FROM student_progress
)
GROUP BY support_status
ORDER BY support_status DESC;
```

Output

Results | Explain | Describe | Saved SQL | History

SUPPORT_STATUS	TOTAL_STUDENTS
On Track	16
Needs Support	9

2 rows returned in 0.03 seconds CSV Export

PL/SQL Function 1: Academic Risk Category of a Student

```
CREATE OR REPLACE FUNCTION fn_risk_category (p_id IN NUMBER)
RETURN VARCHAR2 IS
  v_gpa student_progress.gpa%TYPE;
  v_att student_progress.attendance_pct%TYPE;
  v_back student_progress.backlogs%TYPE;
BEGIN
  IF p_id IS NULL OR p_id <= 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'Invalid student ID');
  END IF;
  SELECT gpa, attendance_pct, backlogs
  INTO v_gpa, v_att, v_back
  FROM student_progress
  WHERE student_id = p_id;
  IF v_gpa < 5.0 OR v_att < 75 OR v_back >= 1 THEN
    RETURN 'Needs Support';
  END IF;
  RETURN 'On Track';
```

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20001,
      'Student ID ' || p_id || ' not found');
END fn_risk_category;
/
```

Results Explain Describe Saved SQL History

```
Function created.

0.17 seconds
```

Execution

```
SELECT fn_risk_category(104) FROM dual;
SELECT fn_risk_category(116) FROM dual;
```

Output

Results Explain Describe Saved SQL History

FN_RISK_CATEGORY(104)
Needs Support

1 rows returned in 0.02 seconds CSV Export

Results Explain Describe Saved SQL History

FN_RISK_CATEGORY(116)
On Track

1 rows returned in 0.02 seconds CSV Export

fn_risk_category(104) returns Needs Support and fn_risk_category(116) returns On Track.

PL/SQL Procedure 2: Academic Progress Report of a Selected Student

```
CREATE OR REPLACE PROCEDURE sp_student_report (p_id IN NUMBER) IS
```

```
 r student_progress%ROWTYPE;
BEGIN
 IF p_id IS NULL OR p_id <= 0 THEN
  RAISE_APPLICATION_ERROR(-20002, 'Invalid student ID');
 END IF;
 SELECT * INTO r FROM student_progress WHERE student_id = p_id;
 DBMS_OUTPUT.PUT_LINE('Student : ' || r.student_id || ' - ' || r.student_name);
 DBMS_OUTPUT.PUT_LINE('Semester : ' || r.semester);
 DBMS_OUTPUT.PUT_LINE('GPA : ' || r.gpa);
 DBMS_OUTPUT.PUT_LINE('Attendance : ' || r.attendance_pct || '%');
 DBMS_OUTPUT.PUT_LINE('Internal : ' || r.internal_marks || '/25');
 DBMS_OUTPUT.PUT_LINE('Backlogs : ' || r.backlogs);
 DBMS_OUTPUT.PUT_LINE('Status : ' || fn_risk_category(p_id));
 IF r.gpa < 5.0 THEN
  DBMS_OUTPUT.PUT_LINE('Reason : GPA below 5.0');
 END IF;
 IF r.attendance_pct < 75 THEN
  DBMS_OUTPUT.PUT_LINE('Reason : Attendance below 75%');
 END IF;
 IF r.backlogs >= 1 THEN
  DBMS_OUTPUT.PUT_LINE('Reason : Has backlogs');
 END IF;
EXCEPTION
 WHEN NO_DATA_FOUND THEN
  DBMS_OUTPUT.PUT_LINE('Error: Student ID ' || p_id || ' not found.');
 WHEN OTHERS THEN
  DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END sp_student_report;
/
```

Results Explain Describe Saved SQL History

```
Procedure created.

0.03 seconds
```

Execution

```
SET SERVEROUTPUT ON
EXEC sp_student_report(104);
```

Output

Results Explain Describe Saved SQL History

```
Student : 104 - Deepika M
Semester : 1
GPA : 3
Attendance : 62.5%
Internal : 10/25
Backlogs : 2
Status : Needs Support
Reason : GPA below 5.0
Reason : Attendance below 75%
Reason : Has backlogs

Statement processed.

0.00 seconds
```

PL/SQL Procedure 3: Semester-wise Student Support Report

```
CREATE OR REPLACE PROCEDURE sp_semester_support_report (p_sem IN NUMBER)
IS
  v_total NUMBER := 0;
  v_need NUMBER := 0;
  e_bad_sem EXCEPTION;
  e_no_data EXCEPTION;
BEGIN
  IF p_sem IS NULL OR p_sem NOT BETWEEN 1 AND 8 THEN
    RAISE e_bad_sem;
  END IF;
  SELECT COUNT(*) INTO v_total
  FROM student_progress WHERE semester = p_sem;
  IF v_total = 0 THEN
    RAISE e_no_data;
  END IF;
  DBMS_OUTPUT.PUT_LINE('Semester ' || p_sem || ' - Students Needing Support');
  DBMS_OUTPUT.PUT_LINE('ID   NAME          GPA   ATT%   BACKLOGS');
  FOR s IN (SELECT * FROM student_progress
     WHERE semester = p_sem ORDER BY student_id) LOOP
    IF fn_risk_category(s.student_id) = 'Needs Support' THEN
      v_need := v_need + 1;
      DBMS_OUTPUT.PUT_LINE(s.student_id || ' ' || RPAD(s.student_name, 13) ||
```

```
      TO_CHAR(s.gpa, '0.0') || ' ' ||
      TO_CHAR(s.attendance_pct, '990.0') || ' ' || s.backlogs);
   END IF;
 END LOOP;
 DBMS_OUTPUT.PUT_LINE('Total: ' || v_total || ' | Needs Support: ' || v_need ||
   ' | On Track: ' || (v_total - v_need));
EXCEPTION
 WHEN e_bad_sem THEN
   DBMS_OUTPUT.PUT_LINE('Error: Semester must be between 1 and 8.');
 WHEN e_no_data THEN
   DBMS_OUTPUT.PUT_LINE('No student records found for semester ' || p_sem);
 WHEN OTHERS THEN
   DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END sp_semester_support_report;
/
```

Results Explain Describe Saved SQL History

```
Procedure created.

0.03 seconds
```

Execution

```
EXEC sp_semester_support_report(1);
```

Output

Results Explain Describe Saved SQL History

```
Semester 1 - Students Needing Support
ID   NAME           GPA   ATT%   BACKLOGS
102 Bhavani S     5.0   78.0 1
104 Deepika M     3.0   62.5 2
106 Farhana B     4.5   74.0 1
109 Imran L       3.5   68.0 1
Total: 12 | Needs Support: 4 | On Track: 8

Statement processed.

0.00 seconds
```

EXEC sp_semester_support_report(2);

Output

Results Explain Describe Saved SQL History

```
Semester 2 - Students Needing Support
ID   NAME           GPA   ATT%   BACKLOGS
113 Meena A       2.5   58.0 3
117 Ramesh H      4.0   72.5 1
120 Uma P         4.5   70.0 0
122 Yamini B      3.0   64.0 2
124 Abirami C     6.0   80.0 1
Total: 13 | Needs Support: 5 | On Track: 8

Statement processed.

0.00 seconds
```

Across both semesters this gives 9 Needs Support and 16 On Track, which matches Query 5.

Exception Handling

Fifteen test cases were executed to check normal, boundary and error behaviour.

#	Test	Input	Expected result
1	Valid, at risk	fn_risk_category(104)	Needs Support
2	Valid, safe	fn_risk_category(116)	On Track
3	Boundary: GPA exactly 5.0	fn_risk_category(115)	On Track (5.0 is not below 5.0)
4	Backlog only	sp_student_report(124)	Needs Support, reason: Has backlogs
5	GPA and attendance only	sp_student_report(120)	Needs Support, reasons: GPA below 5.0 and attendance below 75%
6	Missing record (procedure)	sp_student_report(999)	Error: Student ID 999 not found.
7	Missing record (function)	SELECT fn_risk_category(999) FROM dual	ORA-20001: Student ID 999 not found
8	Invalid input	sp_student_report(-5)	Error: ORA-20002: Invalid student ID
9	NULL input	sp_student_report(NULL)	Error: ORA-20002: Invalid student ID
10	Valid semester	sp_semester_support_report(2)	5 students listed, Total 13
11	Invalid semester	sp_semester_support_report(9)	Error: Semester must be between 1 and 8.
12	Semester with no data	sp_semester_support_report(5)	No student records found for semester 5
13	Duplicate primary key	INSERT with student_id 101	ORA-00001: unique constraint (PK_STUDENT_PROGRESS) violated
14	Out-of-range GPA	INSERT with gpa 11.0	ORA-02290: check constraint (CK_SP_GPA) violated
15	Out-of-range attendance	INSERT with attendance_pct 105	ORA-02290: check constraint (CK_SP_ATTENDANCE) violated

Test Case 1 – Valid, at risk

SELECT fn_risk_category(104) FROM dual;

Output

Results Explain Describe Saved SQL History

FN_RISK_CATEGORY(104)
Needs Support

1 rows returned in 0.02 seconds CSV Export

Test Case 2 – Valid, safe

SELECT fn_risk_category(116) FROM dual;

Output

Results Explain Describe Saved SQL History

FN_RISK_CATEGORY(116)
On Track

1 rows returned in 0.02 seconds CSV Export

Test Case 3 – Boundary: GPA exactly 5.0

SELECT fn_risk_category(115) FROM dual;

Output

Results Explain Describe Saved SQL History

FN_RISK_CATEGORY(115)
On Track

1 rows returned in 0.02 seconds CSV Export

Test Case 4 – Backlog only

```
BEGIN
  sp_student_report(124);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Student : 124 - Abirami C
Semester : 2
GPA : 6
Attendance : 80%
Internal : 16/25
Backlogs : 1
Status : Needs Support
Reason : Has backlogs

Statement processed.

0.00 seconds
```

Test Case 5 – GPA and attendance only

```
BEGIN
  sp_student_report(120);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Student : 120 - Uma P
Semester : 2
GPA : 4.5
Attendance : 70%
Internal : 13/25
Backlogs : 0
Status : Needs Support
Reason : GPA below 5.0
Reason : Attendance below 75%

Statement processed.

0.00 seconds
```

Test Case 6 – Missing record (procedure)

```
BEGIN
  sp_student_report(999);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Error: Student ID 999 not found.

Statement processed.

0.00 seconds
```

Test Case 7 – Missing record (function)

```
SELECT fn_risk_category(999) FROM dual;
```

Output

Results Explain Describe Saved SQL History

```
ORA-20001: Student ID 999 not found
ORA-06512: at "SWETHA.FN_RISK_CATEGORY", line 18

0.00 seconds
```

Test Case 8 – Invalid input

```
BEGIN
  sp_student_report(-5);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Error: ORA-20002: Invalid student ID

Statement processed.

0.00 seconds
```

Test Case 9 – NULL input

```
BEGIN
  sp_student_report(NULL);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Error: ORA-20002: Invalid student ID

Statement processed.

0.00 seconds
```

Test Case 10 – Valid semester

```
BEGIN
  sp_semester_support_report(2);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Semester 2 - Students Needing Support
ID   NAME           GPA   ATT%   BACKLOGS
113 Meena A       2.5   58.0 3
117 Ramesh H      4.0   72.5 1
120 Uma P         4.5   70.0 0
122 Yamini B      3.0   64.0 2
124 Abirami C     6.0   80.0 1
Total: 13 | Needs Support: 5 | On Track: 8

Statement processed.

0.00 seconds
```

Test Case 11 – Invalid semester

```
BEGIN
  sp_semester_support_report(9);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Error: Semester must be between 1 and 8.

Statement processed.

0.00 seconds
```

Test Case 12 – Semester with no data

```
BEGIN
  sp_semester_support_report(5);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
No student records found for semester 5

Statement processed.

0.00 seconds
```

Test Case 13 – Duplicate primary key

```
BEGIN
  INSERT INTO student_progress
  VALUES (101,'Test Student','GP','F',17,1,3,0,10,10,10,15,80.00,0,5.0);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
ORA-00001: unique constraint (PK_STUDENT_PROGRESS) violated

0.00 seconds
```

Test Case 14 – Out-of-range GPA

```
BEGIN
  INSERT INTO student_progress
  VALUES (126,'Test Student','GP','F',17,1,3,0,10,10,10,15,80.00,0,11.0);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
ORA-02290: check constraint (CK_SP_GPA) violated

0.00 seconds
```

Test Case 15 – Out-of-range attendance

```
BEGIN
  INSERT INTO student_progress
  VALUES (127,'Test Student','GP','F',17,1,3,0,10,10,10,15,105.00,0,5.0);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
ORA-02290: check constraint (CK_SP_ATTENDANCE) violated

0.00 seconds
```

Test 3 shows the boundary rule from Query 5: student 115 has GPA 5.0, attendance 81% and no backlogs, so the student is On Track. Tests 13 to 15 show that invalid data is rejected by the table constraints before any PL/SQL runs.

Dataset Insights

- The dataset contains 25 student records: 12 in semester 1 and 13 in semester 2.
- The average GPA is 6.04 for semester 1 and 5.85 for semester 2.
- Nine students (36%) are flagged Needs Support and sixteen are On Track.
- Eight students have one or more backlogs; Meena A (ID 113) has the most (3 backlogs), the lowest GPA (2.5) and the lowest attendance (58%).
- Uma P (ID 120) has no backlogs but is flagged because GPA (4.5) and attendance (70%) are both below the thresholds.
- Priya G (ID 116) has the highest GPA (9.5) and Gayathri V (ID 107) has 100% attendance.
- The same support rule is used in Query 5 and in the PL/SQL function, so SQL and PL/SQL results agree.

Result

Thus, the Student Academic Progress and Early Support system was successfully implemented using SQL and PL/SQL. The STUDENT_PROGRESS table was created, student records were inserted, SQL queries were executed successfully, PL/SQL functions and procedures were implemented, and exception handling was tested successfully.

Reference: Cortez, P. and Silva, A. (2008). Using data mining to predict secondary school student performance. UCI Machine Learning Repository, Student Performance dataset.